

PKCE OAuth with DPoP and Keycloak integration ▾

create simple html with javascript to use pkce oauth + dpop connecting keycloak running at local host 7000 .Html running at 5005 .While accessing html it will route to login and while login to key clock it will sent depop in header.After succesfully login it will show hell0 and store token with depop at local storgae

```
"issuer": "http://localhost:7000/realms/dpop-test", "authorization_endpoint":
"http://localhost:7000/realms/dpop-test/protocol/openid-
connect/auth", "token_endpoint": "http://localhost:7000/realms/dpop-
test/protocol/openid-connect/token", "introspection_endpoint":
"http://localhost:7000/realms/dpop-test/protocol/openid-
connect/token/introspect", "userinfo_endpoint":
"http://localhost:7000/realms/dpop-test/protocol/openid-
connect/userinfo", "end_session_endpoint":
"http://localhost:7000/realms/dpop-test/protocol/openid-connect/logout",
```

Show less

11:09 ↺ ✎ 📄

Viewed a file, created a file, read a file >

Here's your complete single-file PKCE + DPoP client. Here's everything it does and what you need to configure:

What the file does

1. **On load** — checks `localStorage` for existing tokens; if found, shows the Hello screen immediately
2. **Login button** — generates a PKCE code verifier/challenge + random state, pre-generates an EC P-256 DPoP keypair, saves both to `sessionStorage`, then redirects to Keycloak
3. **Callback** — Keycloak redirects back with `?code=...&state=...`; the page creates a fresh DPoP proof (`POST` to the token endpoint) and exchanges the code
4. **Token storage** — stores `access_token`, `refresh_token`, `id_token`, `expires_in`, and the DPoP public JWK in `localStorage` under `auth_tokens`
5. **Hello screen** — displays all stored token details and a sign-out button

Before running — one required change

In the `CONFIG` block at the top, set your actual Keycloak client ID:

```
js
client_id: 'dpop-client', // ← replace with your client ID
```

Keycloak client settings required

Setting	Value
Client authentication	OFF (public client)
Standard flow	Enabled
Valid redirect URIs	<code>http://localhost:5005/*</code>
Web origins	<code>http://localhost:5005</code>
DPoP-bound access tokens	Enabled (in Advanced tab)

Serve the file

```
bash
# Python (simplest)
cd /path/to/folder
python3 -m http.server 5005

# or Node
npx serve -p 5005 .
```

Then open `http://localhost:5005` — it will immediately redirect to Keycloak login.



Index

Code · HTML

Download



How it defer from bearer token

Looking at your JWT, I can see the key difference right there in the **cnf** claim at the bottom:

```
json
{
  "cnf": {
    "jkt": "o840jpUy1348UF1E33GnLmu2ImLNeD9Hgny54yd1xKA"
  }
}
```

DPoP Token vs Bearer Token

Bearer Token — "Possession is 9/10 of the law"

Client —[token]—> Server ☒ accepted
 Attacker steals token
 Attacker —[same token]—> Server ☒ also accepted ❌

A Bearer token is like **cash** — whoever holds it can spend it. No proof of identity needed.

Your DPoP Token — Proof of Possession

Client —[token + DPoP proof signed with private key]—> Server ☒
 Attacker steals token
 Attacker —[same token, no private key]—> Server ❌ REJECTED

DPoP is like a **debit card + PIN**. The card alone is useless without the PIN.

What makes your token DPoP-bound

Claim	Your value	What it means
typ	DPoP	Token type explicitly marked as DPoP
cnf.jkt	o840jpU...	SHA-256 thumbprint of your EC P-256 public key
azp	dpop-client	Client that holds the matching private key

The `cnf.jkt` is the critical field. Keycloak **embedded your DPoP public key's thumbprint** into the token at issuance. Every resource server that receives this token must verify that the `DPoP` header proof was signed by the private key whose public key hashes to `o840jpU...`.

Per-request proof (what happens on every API call)

```
Bearer:  GET /api  Authorization: Bearer <token>
          ↑ same header every time, replayable anywhere

DPoP:    GET /api  Authorization: DPoP <token>
          DPoP: <fresh JWT signed with private key>
                  containing htm=GET, htu=https://api/resource, iat=now
          ↑ proof is tied to THIS method + THIS URL + THIS second
```

The DPoP proof is **single-use, URL-bound, and time-bound** — even if an attacker captures both the token and a proof, replaying it against a different endpoint or a minute later will fail.

What token sent to resource server to validate request

Looking at your JWT image, here's exactly what gets sent:

Two things are sent together — always

```
Authorization: DPoP <access_token>
DPoP: <fresh_proof_jwt>
```

1. The Access Token (from localStorage)

This is the token in your image — the one with `typ: DPoP`, `cnf.jkt: o840jpU...`. It's **static** — same token until it expires.

It proves: WHO you are (sub: 3f77cc34-215b...)
WHICH client (azp: dpop-client)
WHICH key is allowed (cnf.jkt: o840jpU...)

2. The DPoP Proof (generated fresh every request)

A brand new signed JWT created on the fly for **each request**.

json

Header: { `"typ": "dpop+jwt"`, `"alg": "ES256"`, `"jwk": { ...your public key... }` }

Payload: {

`"jti": "unique-id-never-reused"`,

`"htm": "GET"`, ← HTTP method locked

`"htu": "http://your-api/resource"`, ← URL locked

`"iat": 1748598234` ← timestamp locked (valid ~30s)

}

Signed with: your EC P-256 private key ← only you can produce this

What the Resource Server validates

- Step 1: Verify access_token signature (from Keycloak)
- Step 2: Extract cnf.jkt from token → "o840jpU..."
- Step 3: Extract jwk from DPoP proof header → your public key
- Step 4: Hash that public key → must match cnf.jkt ✓
- Step 5: Verify DPoP proof signature using that public key ✓
- Step 6: Check htm matches actual HTTP method ✓
- Step 7: Check htu matches actual request URL ✓
- Step 8: Check iat is within acceptable window (~30s) ✓
- Step 9: Check jti was never seen before (replay protection) ✓

If any step fails → `401 Unauthorized`

Why stealing just the access token is useless

Attacker has: Authorization: DPoP <stolen_token>
DPoP: ??? ← cannot forge this without the private key

Resource server hits Step 4/5 → REJECTED ❌

The private key **never leaves your browser** — it only exists in the Web Crypto API's non-exportable key store.

I want to call api say getaccount to resource server from html so how I will create request

Great question. Here's exactly how to make a DPoP-bound API call from your HTML:

The Request Builder Function

Add this to your existing HTML:

javascript

```
/* =====  
DPoP-BOUND API CALLER  
===== */  
async function dpopFetch(url, options = {}) {  
  // 1. Load tokens from localStorage  
  const stored = JSON.parse(localStorage.getItem('auth_tokens'));  
  if (!stored?.access_token) throw new Error('Not authenticated');  
  
  // 2. Generate fresh DPoP proof for THIS request  
  const method = (options.method || 'GET').toUpperCase();  
  const dpopProof = await createDpopProof(method, url); // already in your code  
  
  // 3. Build headers  
  const headers = {  
    ...options.headers,  
    'Authorization': `DPoP ${stored.access_token}`,  
    'DPoP': dpopProof,  
  };  
}
```

```
'Content-Type': 'application/json',  
};  
  
// 4. Fire the request  
const resp = await fetch(url, { ...options, method, headers });  
  
if (resp.status === 401) throw new Error('Unauthorized – token expired or invalid');  
if (!resp.ok) throw new Error(`HTTP ${resp.status}: ${resp.statusText}`);  
  
return resp.json();  
}
```

Call getAccount

javascript

```
async function getAccount() {  
  const RESOURCE_SERVER = 'http://localhost:8080'; // your resource server  
  
  try {  
    setStatus('<span class="spinner"></span>Calling getAccount...', 'info');  
  
    const data = await dpopFetch(`${RESOURCE_SERVER}/api/account`, {  
      method: 'GET',  
    });  
  
    console.log('Account data:', data);  
    showApiResult(data);  
    setStatus('✓ getAccount success', 'info');  
  
  } catch (err) {  
    setStatus(`Error: ${err.message}`, 'error');  
  }  
}
```

What goes over the wire

```
GET /api/account HTTP/1.1  
Host: localhost:8080
```

```
Authorization: DPoP eyJhbGciOiJIUzI1NiJ9...    ← your access token
DPoP: eyJ0eXAiOiJKcG9wK2p3dCIzImFsZyI6...    ← fresh proof (new every call)
Content-Type: application/json
```

Proof payload locked to:

```
htm = "GET"
htu = "http://localhost:8080/api/account"
iat = <now>
jti = <unique, never reused>
```

Add a button to the Hello screen

In your `renderHello()` function, add this button:

javascript

```
<button class="btn btn-primary" onclick="getAccount()" style="margin-bottom:10px">
  ⚡ GET /api/account
</button>
```

And a result display div:

javascript

```
function showApiResult(data) {
  const box = document.getElementById('api-result');
  if (box) box.innerHTML = `
    <div class="token-row">
      <span class="token-key">Response</span>
      <span class="token-val">${JSON.stringify(data, null, 2)}</span>
    </div>`;
}
```

html

```
<!-- Add inside renderHello HTML -->
<div class="token-box" id="api-result" style="min-height:60px">
  - no request made yet -
</div>
```


POST example (create/update)

javascript

```
async function updateAccount(payload) {
  const data = await dpopFetch('http://localhost:8080/api/account', {
    method: 'POST', // proof htm="POST" auto-locked
    body: JSON.stringify(payload),
  });
  return data;
}

// usage
await updateAccount({ name: 'John', email: 'john@example.com' });
```

Key rule

One `createDpopProof(method, url)` call per request. Never reuse a proof — `jti` must be unique and `iat` must be fresh each time. The resource server will reject replays.

Suppose I am creating simple one page fastapi to getaccount request and validate token and return a result. Can u create simple one page fast api

Ran 2 commands, created a file, read a file >

Here's the complete validation flow the server performs:

What it validates (in order)

Request arrives:

Authorization: DPoP <token>

DPoP: <proof>

Step 1 – Access Token

└─ Fetches Keycloak JWKS (cached 5 min)

└─ Verifies RS256/ES256 signature

- └─ Checks issuer = http://localhost:7000/realms/dpop-test
- └─ Checks audience = account
- └─ Checks not expired
- └─ Checks cnf.jkt exists (proves it's DPoP-bound)

Step 2 – DPoP Proof

- └─ typ == "dpop+jwt"
- └─ htm == "GET" (matches actual method)
- └─ htu == "http://localhost:8080/api/account" (matches actual URL)
- └─ iat within 60 seconds (replay protection)
- └─ jti present (unique ID)
- └─ Signature valid using embedded JWK
- └─ SHA-256 thumbprint of embedded JWK == cnf.jkt from token 

Run it

```
bash

# Install deps
pip install fastapi uvicorn python-jose[cryptography] httpx

# Start server
uvicorn main:app --port 8080 --reload
```

One config you may need to change

```
python

AUDIENCE = "account" # must match `aud` in your token
```

From your JWT screenshot, `aud` is `account` — so it should work as-is. If you get a 401 audience error, check what your token's `aud` field actually contains.

Test without the browser (curl)

```
bash

# Will fail – no DPoP
curl http://localhost:8080/api/account
```

```
# → 401: Authorization header must use DPoP scheme
```

```
# Health check (public, no auth)
```

```
curl http://localhost:8080/health
```

```
# → {"status":"ok"}
```



Main
PY

Download

Do Audience is mandatory if yes where to configure

Good question. Yes, audience validation is important for security but let me clarify both sides.

Is Audience Mandatory?

Security-wise: Yes, strongly recommended. Code-wise: You can disable it, but you shouldn't.

Why it matters

Without audience check:

A token issued for "mobile-app" client
can be sent to your "account-api" → ACCEPTED ❌ (wrong)

With audience check:

Same token sent to "account-api"
aud = "mobile-app" ≠ "account-api" → REJECTED ✅ (correct)

The audience prevents **token confusion attacks** — a valid token for one service being replayed against another.

Where to configure in Keycloak (two places)

Place 1 — Client Scope → Add audience mapper

Keycloak Admin Console

- Client Scopes (left sidebar)
- Create client scope OR edit existing
- Mappers tab
- Add mapper → By configuration → Audience

Name	: add-account-audience
Mapper type	: Audience
Included Client Audience	: dpop-client
Add to access token	: ON

Place 2 — Directly on your Client

Keycloak Admin Console

- Clients → dpop-client
- Client Scopes tab
- Add client scope (assign the one you created above)

What your token should look like after

json

```
{
  "aud": ["account", "dpop-client"],    ← dpop-client added
  "azp": "dpop-client",
  "cnf": { "jkt": "o840jpU..." }
}
```

Match it in your FastAPI config

python

```
# main.py - match exactly what's in your token's aud field
AUDIENCE = "account"          # current - works if aud contains "account"

# If you add your client as audience:
AUDIENCE = "dpop-client"      # ← more specific, recommended
```

```
# If aud is a list, jose checks if AUDIENCE is in the list — works fine
```

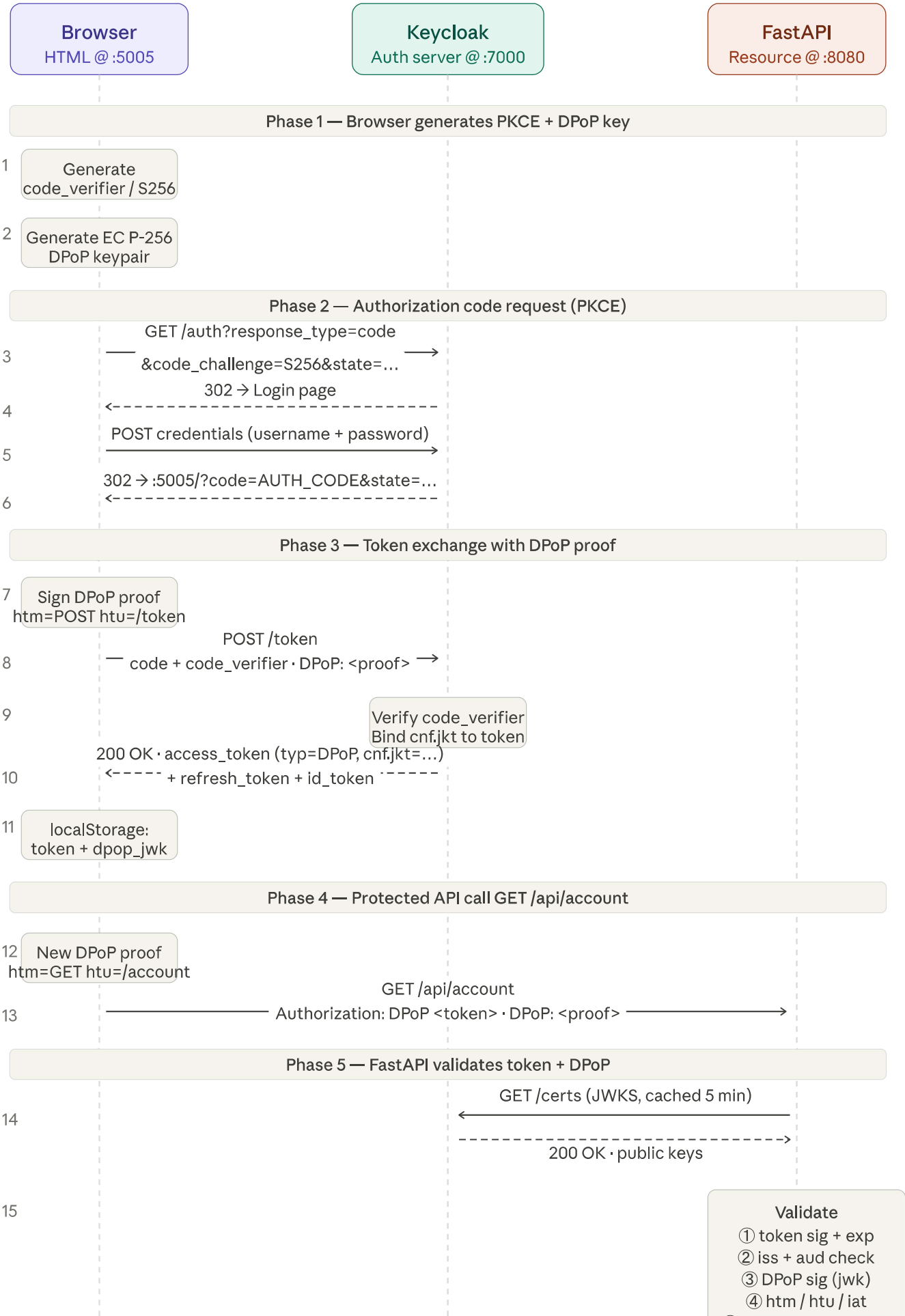
If you want to temporarily disable it (dev only)

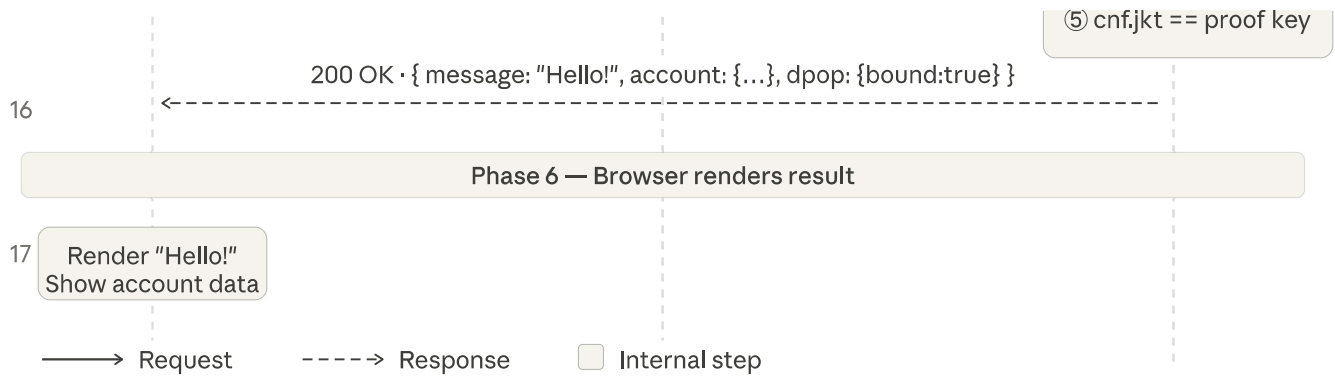
```
python

claims = jwt.decode(
    token,
    key_obj,
    algorithms=["RS256", "ES256"],
    issuer=KEYCLOAK_ISSUER,
    options={
        "verify_exp": True,
        "verify_aud": False,    # ← disable audience check
    },
)
```

Never disable in production — the `cnf.jkt` DPoP binding alone doesn't protect against cross-service token replay.

Can u generate complete flow or seq diag from html to auth and resource server





Here's a walkthrough of each phase:

Phase 1 — Browser setup (before any network call) The browser generates the PKCE `code_verifier` + SHA-256 `code_challenge`, and creates an EC P-256 DPOP keypair stored in Web Crypto. The private key never leaves the browser.

Phase 2 — Auth code request Browser redirects to Keycloak's `/auth` endpoint with the `code_challenge`. User logs in, Keycloak redirects back to `:5005` with a short-lived `AUTH_CODE`.

Phase 3 — Token exchange Browser creates a fresh DPOP proof signed with the private key (`htm=POST`, `htu=/token`), then POSTs it to `/token` alongside the `code` and `code_verifier`. Keycloak verifies the PKCE verifier matches the original challenge, binds the DPOP public key's thumbprint (`cnf.jkt`) into the issued access token, and returns the tokens. These get stored in `localStorage`.

Phase 4 — API call Every API call gets a **brand new** DPOP proof (`htm=GET`, `htu=/api/account`). The request carries both `Authorization: DPOP <token>` and the `DPOP: <proof>` header.

Phase 5 — FastAPI validation (5 checks) FastAPI fetches Keycloak's JWKS (cached), then verifies: ① token signature + expiry, ② issuer + audience, ③ DPOP proof signature using the embedded JWK, ④ `htm` / `htu` / `iat` match the actual request, ⑤ the proof's key thumbprint matches `cnf.jkt` in the token.

Phase 6 — Result Only after all 5 checks pass does the account data flow back. The browser renders "Hello!" with the account details.

